CO3-AT1

Case Study 1 — Smart Medical Diagnosis System

(a) Propositional Logic Representation

Let:

- **F** = Patient has Fever
- **C** = Patient has Cough
- **R** = Patient has Rash
- **B** = Patient has Breathlessness
- **Flu** = Patient has Possible Flu
- **M** = Patient has Possible Measles
- **P** = Patient has Possible Pneumonia

The rules are:

1. **Fever AND Cough → Possible Flu**
2. **Fever AND Rash → Possible Measles**
3. **Cough AND Breathlessness → Possible Pneumonia**

Facts about Patient A:

Symbols Used

Symbol	**Meaning**
	AND
	Implies / If–then
	Fact is true
	Fever
	Cough
	Rash
	Breathlessness
	Possible Flu
	Possible Measles
	Possible Pneumonia

(b) Forward Chaining

Forward Chaining starts with the **known facts** and repeatedly applies rules whose conditions are satisfied.

Initial Facts

Step 1 — Apply Rule I

Rule:

Since:

Therefore:

Derived: Patient A has Possible Flu.

Step 2 — Apply Rule II

Rule:

We know:

but:

Therefore, Rule II **cannot fire**.

No Measles conclusion can be derived.

Step 3 — Apply Rule III

Rule:

Since:

Therefore:

Derived: Patient A has Possible Pneumonia.

Final Forward-Chaining Conclusions

Measles cannot be concluded because there is no Rash fact.

(c) Backward Chaining for Pneumonia

Goal

Look for a rule whose conclusion is Pneumonia.

Rule III:

Therefore, the goal is reduced into two subgoals:

↓

Now check the KB:

Both subgoals are satisfied.

Therefore:

Goal Reduction Tree

```
        Goal: Pneumonia(A)
               |
        Apply Rule III
               |
      Cough(A) AND Breathlessness(A)
            /        \
           /          \
          ✓            ✓
    Cough(A)      Breathlessness(A)
     Fact            Fact
```

Hence:

Case Study 2 — Autonomous Traffic Management Agent

(a) Unification of Rule 1

Rule 1:

Given facts:

We want to match:

with:

Therefore:

Substitution Set / MGU

After substitution:

Both premises are facts, so:

Complete Unification Steps

Step 1

MGU:

Step 2

becomes:

which matches the given fact.

Thus:

and:

is derived.

(b) Resolution Proof of Proceed(CarA)

Step 1 — Convert Rules to CNF

Rule 1

Eliminate implication:

Clause C1:

Rule 2

This gives two clauses:

Rule 3

CNF:

Relevant Facts

Step 2 — Derive ClearPath(Ambulance)

From C1:

Using:

and:

we derive:

Step 3 — Derive GreenSignal(Ambulance)

Using:

and:

we derive:

Step 4 — Negate the Goal

Goal:

For resolution refutation, add its negation:

Step 5 — Apply Rule 3

Rule 3:

Substitute:

Therefore:

Given:

we obtain:

Final Resolution

The derived clause:

and negated goal:

resolve to:

Therefore:

So **CarA can proceed** according to the given knowledge base.

(c) Two Simultaneous Emergency Vehicles

The current KB is **partially sufficient**.

Rule 1 is universally quantified:

Therefore, it can independently process:

and another emergency vehicle such as:

For example:

allows:

and Rule 2 gives:

However, additional information is required

The KB needs facts describing the relationships between the vehicles, such as:

It may also require rules for **conflicting emergency routes**, for example:

and rules to ensure that conflicting green signals are not simultaneously given.

Conclusion

The universal rules can handle **multiple emergency vehicles individually**, but the current KB does not explicitly model **priority, route conflicts, or simultaneous emergencies**. Additional facts and conflict-resolution rules are therefore needed.

Case Study 3 — Agricultural Expert Reasoning System

(a) Conversion to CNF

Given:

P1

Step 1: Eliminate implication

This is already CNF:

P2

Eliminate implication:

Apply De Morgan's law:

Therefore:

P3

Eliminate implication:

Move negation inward:

Therefore:

Facts

These are already CNF unit clauses.

Complete CNF

(b) Resolution Refutation of ApplyDripMethod

Goal

Negate the goal:

Add it to the CNF knowledge base.

Step 1

Clause:

Fact:

Resolution gives:

Step 2

Use P2:

We have:

and:

Resolve with :

Resolve with :

Step 3 — Contradiction

We added the negated goal:

and derived:

Resolution:

gives:

Resolution Proof Tree

```
                 SoilDry
                   |
                   v
        ¬SoilDry ∨ IrrigationNeeded
                   |
                   v
             IrrigationNeeded
                   |
        CropWheat       |
            \       /
             \     /
             P2 clause
              |
              v
         ApplyDripMethod
              |
              |
        ¬ApplyDripMethod
              |
              v
              □
```

Therefore:

(c) Removing SoilDry

Originally:

was used with P1:

Without SoilDry, P1 cannot fire.

Therefore:

We still have:

But P2 requires:

Since IrrigationNeeded is unavailable:

What about CropAtRisk?

P3 says:

The absence of a conclusion that ApplyDripMethod is true **does not mean**:

Therefore, we cannot infer CropAtRisk merely because SoilDry was removed.

So:

Final Result

Knowledge	With SoilDry	Without SoilDry
SoilDry	True	Unknown
IrrigationNeeded	Derived	Not derived
ApplyDripMethod	Derived	Not derived
¬ApplyDripMethod	Not given	Not derived
CropAtRisk	Not derived	Not derived

Important: This demonstrates the difference between **"not provable"** and **"false."**

Case Study 4 — Wumpus World Knowledge Agent

(a) Belief State and Modus Ponens

Percepts

The agent perceives:

These become initial facts in the belief state.

Rule 1

Since:

is true, Modus Ponens gives:

Rule 2

Since:

is true:

Rule 3

Given:

therefore:

Rule 4

No explicit facts of the form:

and:

are provided.

Therefore Rule 4 **cannot fire**.

Derived Knowledge

No specific Safe[x,y] conclusion can be derived.

(b) Forward Chaining — Possible Wumpus and Pit Locations

Assuming the usual four-directional adjacency:

From Stench[1,2]

The cells adjacent to [1,2] are:

Therefore possible Wumpus locations are:

Reasoning:

```
Stench[1,2]
    |
    v
Wumpus is adjacent to [1,2]
    |
    +----> [1,1]
    +----> [1,3]
    +----> [2,2]
```

From Breeze[1,1]

Adjacent cells are:

Therefore:

Reasoning:

```
Breeze[1,1]
    |
    v
Pit is adjacent to [1,1]
    |
    +----> [1,2]
    +----> [2,1]
```

From Glitter[2,2]

Therefore:

Final Possible Locations

Wumpus:

Pit:

Gold:

(c) Is the Path [1,1] → [1,2] → [2,2] Safe?

We examine every cell.

Cell [1,1]

There is:

which indicates that a pit may be adjacent to [1,1].

However, the rule does **not** say that [1,1] itself contains a pit.

So [1,1] is **not proven dangerous**, but it is also not formally proven safe because Rule 4 requires explicit negative knowledge.

Cell [1,2]

We have:

Therefore, a Wumpus is adjacent to [1,2].

Also, from Breeze[1,1]:

Thus [1,2] is a **possible Pit location**.

Therefore:

Cell [2,2]

We know:

However, [2,2] is also adjacent to [1,2], and therefore is among the possible Wumpus locations inferred from the stench:

So [2,2] is **not proven safe**.

Final Path Evaluation

Cell	Knowledge	Safe?
[1,1]	Breeze present; no explicit pit fact	Not proven safe
[1,2]	Possible Pit	**Not safe to assume**
[2,2]	Gold present; possible Wumpus	**Not proven safe**

Therefore:

The agent should **not take this path based on the current KB**, because there is insufficient negative information to establish:

for the cells on the path.